

MemoryLane: Personal Browser Memory Assistant

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Rabeeh Ahmad, CSED

Ishan Singhania, CSED

16 August 2026

1 Higher-order goal

...secondary framing

MemoryLane aims to serve as a personal browser memory assistant that transforms how users interact with previously encountered web content. Rather than relying on limited URL keywords or manually maintained bookmarks, users can retrieve useful information through natural language semantic searches.

While users consume vast amounts of web content daily, current browser systems lack mechanisms to effectively revisit websites without remembering their exact names or URLs. The immediate engineering objective of MemoryLane is to build a deployable browser-to-backend-to-frontend system that captures browsing information, intelligently organizes it, and enables intelligent retrieval through semantic search.

2 Time-to-value

...primary heuristic

- We will deliver a working end-to-end workflow by initially implementing the core pipeline: *browse* $\rightarrow$ *capture* $\rightarrow$ *store* $\rightarrow$ *search*.
- Development will focus on validating the complete system workflow early, starting with webpage capture and storage before adding advanced retrieval capabilities enabling semantic search.
- Success is defined by the ability of a user to retrieve a previously visited webpage using a natural language query without remembering the exact URL, title, or keywords.

3 Problem Statement

Users browse the web extensively and encounter useful information regularly, but often struggle to rediscover it later. Important resources such as articles, documentation, tutorials, and research material are frequently lost because users cannot remember the exact website, title, or keywords required to locate them again.

Common pain points include:

- **Weak recall:** Users often remember the topic or idea of a webpage but not the exact URL, title, or search terms.

- **Browser history limitations:** Standard browser history stores only basic information such as URLs, titles, and timestamps, requiring users to already remember details about what they are searching for.
- **Bookmark friction:** Manually saving and organizing bookmarks requires effort and becomes difficult to maintain as the number of saved resources increases.
- **Search engine mismatch:** General web search retrieves information from the entire internet rather than searching the user’s own previously encountered information.

Why this matters now (contextual relevance): As users consume increasingly diverse and voluminous online content, the gap between discovery and rediscovery grows larger. A personal browsing memory system addresses this challenge by reducing the time spent hunting for previously encountered resources while simultaneously improving information retention and accessibility.

4 Proposed Solution

...Software Proposition

4.1 Overview

MemoryLane is a browser-based personal memory system consisting of three major components:

- **Chrome Extension:** Captures webpage information including URL, title, main content, and timestamp while the user browses.
- **Backend Processing System:** Receives webpage information, extracts meaningful content, generates summaries and searchable representations, manages storage through SQLite for metadata and summaries, and stores semantic representations in ChromaDB.
- **Web Application Interface:** Allows users to search their browsing memories using natural language queries and view retrieved webpages with generated summaries.

4.2 Core workflow

1. The user installs the MemoryLane browser extension and browses normally. The extension captures webpage information such as URL, title, content, and timestamp.
2. The captured webpage data is sent to the backend through REST APIs. The backend processes the webpage by extracting meaningful content and preparing it for storage.
3. The processed webpage content is used to generate:
 - A concise summary using an LLM.
 - A semantic representation (embedding) using a sentence transformer model.
4. The system stores:
 - Metadata and summaries in SQLite.
 - Semantic representations in ChromaDB.
5. The user opens the MemoryLane web application and enters a natural language query.
6. The query is converted into an embedding and compared against stored webpage embeddings to retrieve relevant memories.
7. The system returns the most relevant webpages along with their summaries and metadata.

4.3 Operational constraints for an educational setting

- The system will focus on building a complete working workflow rather than developing new AI models or algorithms.
- Existing libraries and pretrained models will be used for content extraction, embeddings, and summarisation.
- The project will prioritize correctness of the core pipeline: browser capture → processing → storage → retrieval.
- The architecture will remain modular so individual components can be improved independently.

5 Privacy and Data Handling Considerations

5.1 User Consent

The extension should only collect browsing information after explicit user permission. Users should be able to control when MemoryLane is active.

5.2 Sensitive Information Handling

Webpages may contain personal accounts, private documents, and work-related information. Therefore, the system should avoid storing unnecessary information and focus on relevant webpage content.

5.3 Data Storage

For the prototype, stored data is limited to the user's own browsing activity. The system is designed for controlled usage rather than large-scale deployment.

6 Solution Approach

...Engineering focus

This project focuses on implementing an end-to-end information retrieval system for storing information and retrieving it in the future in relatively lesser time using existing tools and models. The approach emphasizes practical system design rather than research on new AI algorithms.

6.1 Content extraction and processing

Webpages contain large amounts of unnecessary information such as advertisements, navigation menus, and unrelated elements.

MemoryLane processes webpages to extract the main useful content.

The extraction pipeline uses:

- **BeautifulSoup:** Used for HTML parsing and webpage element extraction.
- **Mozilla Readability:** Used for extracting the main article or content section from webpages.

For JavaScript-heavy webpages where content is dynamically generated, browser rendering tools such as Playwright or Selenium can be used before extraction (optional).

6.2 Semantic search

MemoryLane uses semantic search to retrieve information based on meaning rather than exact keyword matching.

The process is:

- Extracted webpage content is converted into embeddings using a pretrained Sentence Transformer model.
- User queries are converted into embeddings using the same model.
- Similarity search is performed between the query representation and stored webpage representations.
- The most relevant memories are returned to the user.

The project will use:

- **Sentence Transformers:** For generating webpage and query embeddings.
- **all-MiniLM-L6-v2:** Initial embedding model due to its balance between speed and quality.
- **ChromaDB:** Vector database used for storing embeddings and performing similarity search.

6.3 LLM-based summarisation

MemoryLane uses an LLM as an information organization layer.

The purpose is not to replace the retrieval system, but to make stored memories easier to understand.

For each webpage, the system generates:

- Short summaries.
- Important points from the content.
- Basic categorization information.

This allows users to quickly understand a retrieved memory without opening the original webpage immediately.

6.4 Web architecture

MemoryLane follows a three-component architecture:

- **Frontend:** React application providing the search interface, displaying retrieved memories, and offering real-time control over memory storage settings directly from the browser extension.
- **Backend API:** FastAPI server responsible for webpage ingestion, processing, AI pipeline integration, and search requests.
- **Storage Layer:** SQLite stores structured metadata, while ChromaDB stores semantic representations.

7 Data Flow

The complete data flow is:

1. User visits a webpage.
2. Chrome extension extracts webpage information.
3. Extension sends webpage data to the FastAPI backend.
4. Backend cleans webpage content.
5. The content processing module extracts useful information.
6. LLM generates summaries and important information.
7. Sentence Transformer generates webpage embeddings.
8. SQLite and ChromaDB store the processed information.
9. User enters a query through the MemoryLane website.
10. Query is converted into an embedding.
11. ChromaDB performs similarity search.
12. Relevant webpage memories are returned to the user.

8 Evaluation Criterion

...measurable and attributable

The evaluation of MemoryLane will focus on whether the system successfully enables users to retrieve previously encountered information.

8.1 Memory Retrieval Effectiveness

The primary evaluation criterion will be the ability of the system to retrieve relevant webpages for natural language queries.

Evaluation process:

- Create a collection of previously visited webpages.
- Create test queries describing the information contained in those webpages.
- Evaluate whether the correct or relevant webpage is retrieved.

8.2 Content Extraction Quality

The quality of webpage processing will be evaluated by checking whether the extracted content correctly represents the important information from the original webpage.

8.3 Search Relevance

The semantic search system will be evaluated based on whether retrieved memories match the user's intended query.

8.4 System Usability

The usability of the system will be evaluated by measuring whether users can easily capture webpage memories, search previous information, and understand retrieved results through generated summaries.

9 Scalability

...with foresight

The architecture of MemoryLane separates the frontend, backend, processing pipeline, and storage layers.

This modular design allows individual components to be improved independently.

Possible improvements include:

- Replacing SQLite with PostgreSQL for larger multi-user deployments.
- Improving webpage extraction techniques for more complex websites.
- Supporting larger collections of stored memories.
- Improving retrieval quality with better embedding models.
- Improving the summary of memories using better LLM models.

10 Engine availability heuristic

The project relies on existing technologies and libraries rather than developing new algorithms.

The major components are:

- **Chrome Extension APIs:** Used for browser interaction and webpage capture.
- **React + TypeScript:** Used for frontend development.
- **FastAPI:** Used for backend API development.
- **SQLite:** Used for structured data storage.
- **ChromaDB:** Used for vector storage and semantic retrieval.
- **Sentence Transformers:** Used for generating embeddings.
- **LLM API:** Used for webpage summarisation.
- **BeautifulSoup and Mozilla Readability:** Used for webpage content extraction.

11 Project scope and deliverables

...iteration-friendly

The project will be developed incrementally with the following milestones:

11.1 Initial Prototype

The first working prototype will include:

- Chrome extension capable of capturing webpage information.
- FastAPI backend for receiving and processing webpage data.
- Database storage for webpage memories.
- Basic React interface for searching stored memories.

11.2 Intermediate Deliverables

The system will be extended with:

- Improved webpage content extraction.
- Semantic search using embeddings.
- ChromaDB integration.
- LLM-based summaries and information organization.

11.3 Final Deliverable

The final system will provide:

- Complete browser-to-website workflow.
- Natural language memory search.
- Retrieved webpages with generated summaries.

12 Risks and mitigations

Content Extraction Failures

Risk: Some websites render content dynamically or have complex structures that standard extraction tools cannot handle properly.

Mitigation:

- Defer JavaScript-heavy site support to Phase 2.
- Implement logging for extraction errors and analysis.
- Use robust extraction tools like BeautifulSoup and Mozilla Readability.
- Add browser rendering support using Playwright or Selenium for problematic sites.

Privacy and Data Retention

Risk: Browsing history is sensitive personal information and users need clear control over what is stored.

Mitigation:

- Minimize data collection per user intent.
- Store page content locally where possible.
- Clearly document what is transmitted to backend.
- Provide future local-only processing mode for privacy.

Deployment and Environment Consistency

Risk: System may work reliably in development but encounter failures in production environments due to configuration differences.

Mitigation:

- Use Docker containerization from development through production.

- Establish staging environment for pre-production validation.
- Document all deployment requirements and dependencies.
- Implement CI/CD to ensure repeatable and consistent deployments across environments.

Browser Extension Reliability

Risk: Extension may experience crashes, miss capturing pages, or fail to communicate with backend services.

Mitigation:

- Test extension thoroughly across multiple pages and browser versions.
- Implement comprehensive error logging and reporting in the extension.
- Add graceful fallback mechanisms if backend becomes unavailable.
- Provide users with clear status feedback on capture success and failures.

13 Summary

MemoryLane proposes a deployable three-component system to transform browser history into a searchable personal knowledge base. It meets course criteria by emphasizing end-to-end system design and measurable evaluation metrics focused on retrieval success. By combining browser extension development, backend services, and semantic search, MemoryLane demonstrates a complete information retrieval system that addresses a real user need and grounds engineering workflow integration in practical deployment.